



5.3.3 LU factorization with partial pivoting ¶

05.3.3 LU with row pivoting (right-looking algorithm)

fit width



Having introduced our notation for permutation matrices, we can now define the LU factorization with partial pivoting: Given an $m \times n$ matrix A , we wish to compute

- vector p of n integers that indicates how rows are pivoting as the algorithm proceeds,
- a unit lower trapezoidal matrix L , and
- an upper triangular matrix U

so that $\tilde{P}(p)A = LU$. We represent this operation by

$$[A, p] := \text{LU piv } A,$$

where upon completion A has been overwritten by $\{L \setminus U\}$, which indicates that U overwrites the upper triangular part of A and L is stored in the strictly lower triangular part of A .

Let us start with revisiting the derivation of the right-looking LU factorization in [Subsection 5.2.2](#). The first step is to find a first permutation matrix $\tilde{P}(\pi_1)$ such that the element on the diagonal in the first column is maximal in value. (Mathematically, any nonzero value works. We will see that ensuring that the multiplier is less than one in magnitude reduces the potential for accumulation of error.) For this, we will introduce the function

$$\text{maxi}(x)$$

which, given a vector x , returns the index of the element in x with maximal magnitude (absolute value). The algorithm then proceeds as follows:

- Partition A , L as follows:

$$A \rightarrow \begin{pmatrix} \alpha_{11} & a_{12}^T \\ a_{21} & A_{22} \end{pmatrix}, \quad \text{and} \quad L \rightarrow \begin{pmatrix} 1 & 0 \\ l_{21} & L_{22} \end{pmatrix}$$

- Compute $\pi_1 = \text{maxi}\left(\frac{\alpha_{11}}{a_{21}}\right)$.
- Permute the rows: $\begin{pmatrix} \alpha_{11} & a_{12}^T \\ a_{21} & A_{22} \end{pmatrix} := \tilde{P}(\pi_1) \begin{pmatrix} \alpha_{11} & a_{12}^T \\ a_{21} & A_{22} \end{pmatrix}$.
- Compute $l_{21} := a_{21}/\alpha_{11}$.
- Update $A_{22} := A_{22} - l_{21}a_{12}^T$.

This completes the introduction of zeroes below the diagonal of the first column.

Now, more generally, assume that the computation has proceeded to the point where matrix A has been overwritten by

$$\left(\begin{array}{c|cc} A_{00} & a_{01} & A_{02} \\ \hline 0 & \alpha_{11} & a_{12}^T \\ 0 & a_{21} & A_{22} \end{array} \right)$$

where A_{00} is upper triangular. If no pivoting was added one would compute $l_{21} := a_{21}/\alpha_{11}$ followed by the update

$$\left(\begin{array}{c|cc} A_{00} & a_{01} & A_{02} \\ \hline 0 & \alpha_{11} & a_{12}^T \\ 0 & a_{21} & A_{22} \end{array} \right) := \left(\begin{array}{c|cc} I & 0 & 0 \\ \hline 0 & 1 & 0 \\ 0 & -l_{21} & I \end{array} \right) \left(\begin{array}{c|cc} A_{00} & a_{01} & A_{02} \\ \hline 0 & \alpha_{11} & a_{12}^T \\ 0 & a_{21} & A_{22} \end{array} \right) = \left(\begin{array}{c|cc} A_{00} & a_{01} & A_{02} \\ \hline 0 & \alpha_{11} & a_{12}^T \\ 0 & 0 & A_{22} - l_{21}a_{12}^T \end{array} \right).$$

Now, instead one performs the steps

- Compute

$$\pi_1 := \text{maxi}\left(\frac{\alpha_{11}}{a_{21}}\right).$$

- Permute the rows:

$$\left(\begin{array}{c|cc} A_{00} & a_{01} & A_{02} \\ \hline 0 & \alpha_{11} & a_{12}^T \\ 0 & a_{21} & A_{22} \end{array} \right) := \left(\begin{array}{c|c} I & 0 \\ \hline 0 & \tilde{P}(\pi_1) \end{array} \right) \left(\begin{array}{c|cc} A_{00} & a_{01} & A_{02} \\ \hline 0 & \alpha_{11} & a_{12}^T \\ 0 & a_{21} & A_{22} \end{array} \right)$$

- Update

$$l_{21} := a_{21}/\alpha_{11}.$$

- Update

$$\left(\begin{array}{c|cc} A_{00} & a_{01} & A_{02} \\ \hline 0 & \alpha_{11} & a_{12}^T \\ 0 & a_{21} & A_{22} \end{array} \right) := \left(\begin{array}{c|cc} I & 0 & 0 \\ \hline 0 & 1 & 0 \\ 0 & -l_{21} & I \end{array} \right) \left(\begin{array}{c|cc} A_{00} & a_{01} & A_{02} \\ \hline 0 & \alpha_{11} & a_{12}^T \\ 0 & a_{21} & A_{22} \end{array} \right)$$

$$= \left(\begin{array}{c|cc} A_{00} & a_{01} & A_{02} \\ \hline 0 & \alpha_{11} & a_{12}^T \\ 0 & 0 & A_{22} - l_{21}a_{12}^T \end{array} \right).$$

This algorithm is summarized in [Figure 5.3.3.1](#). In that algorithm, the lower triangular matrix L is accumulated below the diagonal.

```

[A, p] = LUPiv-right-looking(A)
A → ( ATL | ATR
      ABL | ABR ), p → ( pT
                        pB )
ATL is 0 × 0, pT has 0 elements
while n(ATL) < n(A)
    ( ATL | ATR
      ABL | ABR ) → ( A00 | a01 A02
                      a10T | α11 a12T
                      A20 | a21 A22 ), ( pT
                                         pB ) → ( p0
                                                  π1
                                                  p2 )
    π1 := maxi ( α11 / a21 )
    ( A00 | a01 A02
      a10T | α11 a12T
      A20 | a21 A22 ) := ( I | 0
                             0 | P(π1) ) ( A00 | a01 A02
                                             a10T | α11 a12T
                                             A20 | a21 A22 )
    a21 := a21/α11
    A22 := A22 - a21a12T
    ( ATL | ATR
      ABL | ABR ) ← ( A00 a01 | A02
                       a10T α11 | a12T
                       A20 a21 | A22 ), ( pT
                                         pB ) ← ( p0
                                                  π1
                                                  p2 )
endwhile

```

Figure 5.3.3.1. Right-looking LU factorization algorithm with partial pivoting.

05.3.3 LU factorization with pivoting: PA = L U

fit width



What this algorithm computes is a sequence of Gauss transforms $L_0, \dots, L_{n-1}$ and permutations $P_0, \dots, P_{n-1}$ such that

$$L_{n-1}P_{n-1} \cdots L_0P_0A = U$$

or, equivalently,

$$A = P_0^T L_0^{-1} \cdots P_{n-1}^T L_{n-1}^{-1} U.$$

Actually, since $P_k = \left(\begin{array}{c|c} I_{k \times k} & 0 \\ \hline 0 & \tilde{P}(\pi) \end{array} \right)$ for some π , we know that $P_k^T = P_k$ and hence

$$A = P_0 L_0^{-1} \cdots P_{n-1} L_{n-1}^{-1} U.$$

What we will finally show is that there are Gauss transforms $L_0^*, \dots, L_{n-1}^*$ such that

$$A = P_0 \cdots P_{n-1} \underbrace{L_0^* \cdots L_{n-1}^*}_L U$$

or, equivalently,

$$\tilde{P}(p)A = P_{n-1} \cdots P_0 A = \underbrace{L_0^* \cdots L_{n-1}^*}_L U,$$

which is what we set out to compute.

Here is the insight. If only we knew how to order the rows of A and right-hand side b correctly, then we would not have to pivot. But we only know how to pivot as the computation unfolds. Recall that the multipliers can overwrite the elements they zero in Gaussian elimination and do so when we formulate it as an LU factorization. By not only pivoting the elements of

$$\left(\begin{array}{cc} \alpha_{11} & a_{12}^T \\ a_{21} & A_{22} \end{array} \right)$$

but also all of

$$\left(\begin{array}{c|cc} a_{10}^T & \alpha_{11} & a_{12}^T \\ A_{20} & a_{21} & A_{22} \end{array} \right),$$

we are moving the computed multipliers with the rows that are being swapped. It is for this reason that we end up computing the LU factorization of the permuted matrix: $\tilde{P}(p)A$.

Homework 5.3.3.1. Implement the algorithm given in [Figure 5.3.3.1](#) as

```
function [ A_out ] = LUpiv_right_looking( A )
```

by completing the code in [Assignments/Week05/matlab/LUpiv_right_looking.m](#). Input is an $m \times n$ matrix A . Output is the matrix A that has been overwritten by the LU factorization and pivot vector p . You may want to use [Assignments/Week05/matlab/test_LUpiv_right_looking.m](#) to check your implementation.

The following utility functions may come in handy:

- [Assignments/Week05/matlab/maxi.m](#)
- [Assignments/Week05/matlab/Swap.m](#)

which we hope are self explanatory.

▼ [Solution](#)

See [Assignments/Week05/answers/LUpiv_right_looking.m](#).
(Assignments/Week05/answers/LUpiv_right_looking.m)